

Soccer Forecast Agent — Progress Report

Progress Report

The Soccer Forecast Agent is a multi-agent AI system that monitors Premier League fixtures, computes probabilistic forecasts for the match winner and over/under 2.5 goals markets, and sends email alerts when a statistically meaningful edge is detected relative to market odds. Phases 1 through 3 of an eight-phase roadmap are complete, with Phase 4 in progress.

Framework familiarity. The system is built on LangGraph for multi-agent orchestration, Python for all agent and analytics logic, SQLite for structured persistence, and ChromaDB for vector retrieval. Provider adapters for both OpenAI and Anthropic Claude are implemented and swappable via a single environment variable.

Prompting experiments. The ReAct research agent uses a YAML-backed prompt module that separates prompt text from orchestration logic, making it possible to tune the system prompt and evidence-extraction prompt independently without touching agent code. Two distinct prompt templates are maintained: one for the research loop (think-act-observe) and one for structured evidence extraction from raw LLM output. Prompt injection from untrusted web content is handled by a dedicated content guard.

Multi-agent orchestration. The architecture follows a supervisor-based pattern with four agents: StatsMarketAgent (complete), NewsContextAgent (complete), SynthesisAlertAgent (in progress), and SupervisorAgent (planned via LangGraph StateGraph with typed shared state). Agents communicate through a shared GraphState TypedDict and depend only on injected Protocol interfaces.

Memory. Two memory layers are implemented. SQLite stores structured data — fixtures, forecasts, evidence items, source reliability scores, and user preferences — via a repository pattern with five separate protocol interfaces. ChromaDB stores embedded article chunks for semantic retrieval, enabling the ReAct agent to seed its research loop with relevant prior context before issuing live web searches.

Tool use. Tools are exposed through an MCP server: a fixture fetcher (football-data.org), an odds fetcher (The Odds API with bookmaker ranking), and a web search tool (Tavily). An article ingester chunks and embeds search results into ChromaDB. An SMTP email alert channel handles delivery. The ReAct agent dispatches tool calls through a ToolDispatcher that routes to the MCP client, keeping tool implementation decoupled from agent logic.

Safeguards. An AlertGuard enforces six checks before any alert fires: minimum evidence count, minimum average source reliability, maximum evidence age, minimum source diversity, minimum edge threshold, and spam suppression within a configurable time window. A content guard wraps all external web content to defend against prompt injection. Every alert payload carries a mandatory conservative framing disclaimer.

System Architecture (Planned + Partially Implemented)

The system is designed as a supervisor-based multi-agent pipeline with four agents orchestrated by LangGraph. All agents depend on abstract Protocol interfaces — no agent imports a concrete SDK directly.

The **SupervisorAgent** (Phase 5, planned) manages the workflow and routes shared state between the three worker agents. The **StatsMarketAgent** (Phase 2, complete) fetches upcoming fixtures and market odds from external APIs and computes a statistical baseline forecast from historical SQLite data. The **NewsContextAgent** (Phase 3, complete) runs a ReAct research loop that seeds context from a ChromaDB vector store, issues live web searches via Tavily, and extracts structured evidence items using the LLM. The **SynthesisAlertAgent** (Phase 4, in progress) will combine the baseline and evidence using a log-odds adjustment mechanism, evaluate the result against the AlertGuard thresholds, and dispatch an email alert if the edge and confidence requirements are met.

Structured data (fixtures, forecasts, evidence, source reliability scores) is stored in SQLite via a repository pattern. Unstructured news and article content is embedded and stored in ChromaDB for semantic retrieval.

Implementation Status

Component	Status	Phase
Data models, repository protocols	Complete	1
LLM + embedding providers (OpenAI, Claude)	Complete	1
Fixtures, odds, search, ingester tools	Complete	1
SQLite + ChromaDB repositories	Complete	1
Alert guardrails, email sender, MCP server	Complete	1
StatsMarketAgent — baseline analytics	Complete	2
NewsContextAgent — ReAct research loop	Complete	3
SynthesisAlertAgent — synthesis + alert	In progress	4

Component	Status	Phase
SupervisorAgent — LangGraph orchestration	Planned	5

Key Design Principles

Principle	Implementation
Single Responsibility	Each class has one job — agents orchestrate; tools fetch; repositories persist
Open/Closed	New LLM providers, embedding models, or alert channels can be added without modifying existing code
Dependency Inversion	All agents receive injected Protocol interfaces — no concrete SDK imports in agent logic
Interface Segregation	Five separate repository protocols: MatchRepository, ForecastRepository, EvidenceRepository, SourceReliabilityRepository, VectorRepository

Phase 1 — Core Infrastructure

All data models, repository protocols, LLM and embedding provider adapters, API tools, guardrails, and the MCP server. Every layer is typed and injectable.

```
In [1]: # -- Data Models: real source from soccer_forecast_agent/models/ -----
import inspect, sys, os
sys.path.insert(0, os.path.abspath("."))
from dotenv import load_dotenv
load_dotenv()

from soccer_forecast_agent.models.match import (
    Match, MarketOdds, BaselineForecast, MatchContext,
)
from soccer_forecast_agent.models.evidence import EvidenceItem

print("=== match.py: key models ===")
print()
for cls in (Match, MarketOdds, BaselineForecast):
    print(inspect.getsource(cls))
    print()
```

```
=== match.py: key models ===
```

```
@dataclass
class Match:
    """A Premier League fixture with its current status and optional final score."""

    match_id: str
    competition: str
    home_team: str
    away_team: str
    kickoff_time: datetime
    status: str # "upcoming" | "live" | "resolved"
    final_score: str | None = None
```

```
@dataclass
class MarketOdds:
    """Decimal odds for a match across winner and over/under markets at a point in time."""

    odds_id: str
    match_id: str
    timestamp: datetime
    home_win: float
    draw: float
    away_win: float
    over_2_5: float
    under_2_5: float
```

```
@dataclass
class BaselineForecast:
    """Probabilities produced by the statistical baseline, independent of market odds."""

    home_win: float
    draw: float
    away_win: float
    over_2_5: float
    under_2_5: float
```

```
In [2]: # -- Repository Protocols: real source from soccer_forecast_agent/ -----
from soccer_forecast_agent.providers.llm import LLMProvider
from soccer_forecast_agent.memory.repository import (
    MatchRepository, VectorRepository,
)

print("=== LLMProvider protocol ===")
print(inspect.getsource(LLMProvider))
print()
print("=== MatchRepository protocol ===")
print(inspect.getsource(MatchRepository))
print()
```

```
print("=== VectorRepository protocol ===")
print(inspect.getsource(VectorRepository))
print()
print("Concrete: SQLiteRepository, ChromaVectorRepository")
print("LLM adapters: OpenAIProvider, ClaudeProvider")
```

```

=== LLMProvider protocol ===
class LLMProvider(Protocol):
    """Abstract interface for language model calls. Agents depend on this, n
ever on a concrete SDK."""

    def chat(self, messages: list[dict[str, str]], **kwargs: Any) -> str:
        """Send a conversation and return the assistant reply as a strin
g."""
        ...

    def chat_with_tools(
        self,
        messages: list[dict[str, str]],
        tools: list[dict],
        **kwargs: Any,
    ) -> dict:
        """Send a conversation with tool definitions; return the raw respons
e dict including any tool calls."""
        ...

=== MatchRepository protocol ===
class MatchRepository(Protocol):
    """Persistence contract for match fixtures."""

    def save_match(self, match: Match) -> None:
        """Persist a match, upserting on match_id."""
        ...

    def get_upcoming(self, competition: str) -> list[Match]:
        """Return all matches with status 'upcoming' for the given competi
on."""
        ...

    def get_recent_finished(self, team: str, competition: str, limit: int =
5) -> list[Match]:
        """Return the most recent finished matches for a team in a competi
on."""
        ...

    def update_status(self, match_id: str, status: str, final_score: str | N
one = None) -> None:
        """Update match status and optionally record the final score."""
        ...

=== VectorRepository protocol ===
class VectorRepository(Protocol):
    """Persistence contract for the unstructured news vector store."""

    def upsert(self, chunks: list[ArticleChunk]) -> None:
        """Embed and store article chunks, replacing any with the same chunk
_id."""
        ...

    def search(self, query: str, teams: list[str], top_k: int = 5) -> list[A

```

```
rticleChunk]:
    """Return the top_k most semantically relevant chunks filtered by te
am names."""
    ...
```

Concrete: SQLiteRepository, ChromaVectorRepository
 LLM adapters: OpenAIProvider, ClaudeProvider

```
In [3]: # — Alert Guardrails – real source —
from soccer_forecast_agent.guardrails.alert_guard import AlertGuard, AlertGu

print("=== AlertGuardConfig thresholds ===\n")
print(inspect.getsource(AlertGuardConfig))
print()

config_demo = AlertGuardConfig()
print("Default threshold values:")
print(f"  Min evidence count   : {config_demo.min_evidence_count}")
print(f"  Min avg reliability   : {config_demo.min_avg_reliability}")
print(f"  Max evidence age      : {config_demo.max_evidence_age_hours}h")
print(f"  Min edge threshold    : {config_demo.min_edge_threshold}")
print(f"  Min confidence        : {config_demo.min_confidence_threshold}")
print(f"  Spam window          : {config_demo.spam_window_hours}h")

=== AlertGuardConfig thresholds ===

@dataclass(frozen=True)
class AlertGuardConfig:
    """Tunable thresholds that define the alert quality bar."""

    min_evidence_count: int = 3
    min_avg_reliability: float = 0.5
    max_evidence_age_hours: int = 48
    min_unique_sources: int = 2
    min_edge_threshold: float = 0.05
    min_confidence_threshold: float = 0.60
    spam_window_hours: int = 6
    min_odds_delta: float = 0.05
```

```
Default threshold values:
  Min evidence count   : 3
  Min avg reliability   : 0.5
  Max evidence age      : 48h
  Min edge threshold    : 0.05
  Min confidence        : 0.6
  Spam window          : 6h
```

Phase 2 — Baseline Analytics

The `SimpleBaselineStrategy` derives calibrated match probabilities from recent form, home advantage, and goals averages. **Market odds are deliberately excluded** —

the baseline must be independent of market prices to produce a meaningful edge signal.

```
In [4]: # — Baseline Strategy – real implementation —————  
import inspect, sys, os  
sys.path.insert(0, os.path.abspath("."))  
from dotenv import load_dotenv  
load_dotenv()  
  
from soccer_forecast_agent.analytics.baseline import SimpleBaselineStrategy  
print(inspect.getsource(SimpleBaselineStrategy))
```



```

class SimpleBaselineStrategy:
    """Form and goals-based baseline. No market odds used – baseline must be
    independent of market prices."""

    RESULT_POINTS = {"W": 3, "D": 1, "L": 0}
    MAX_FORM_POINTS = 15 # 5 wins × 3 points
    WIN_PROBABILITY_FLOOR = 0.05

    def __init__(self, home_advantage_boost: float = 0.05) -> None:
        """Initialise with a configurable home advantage scalar added to the
        home team's raw strength."""
        self._home_boost = home_advantage_boost

    def compute(self, context: MatchContext) -> BaselineForecast:
        """Derive probabilities from form, home advantage, and goals average
s."""
        home_form = self._form_score(context.recent_home_results)
        away_form = self._form_score(context.recent_away_results)

        home_attack = context.home_goals_scored_avg
        home_defense = context.home_goals_conceded_avg
        away_attack = context.away_goals_scored_avg
        away_defense = context.away_goals_conceded_avg

        home_strength = (home_form + home_attack) / (1 + home_defense) + self._home_boost
        away_strength = (away_form + away_attack) / (1 + away_defense)

        total = home_strength + away_strength
        raw_home = home_strength / total
        raw_away = away_strength / total

        draw_weight = 0.26
        available_mass = 1 - draw_weight
        raw_home_prob = raw_home * available_mass
        raw_away_prob = raw_away * available_mass
        if raw_away_prob < self.WIN_PROBABILITY_FLOOR:
            away_win = self.WIN_PROBABILITY_FLOOR
            home_win = available_mass - away_win
        elif raw_home_prob < self.WIN_PROBABILITY_FLOOR:
            home_win = self.WIN_PROBABILITY_FLOOR
            away_win = available_mass - home_win
        else:
            home_win, away_win = raw_home_prob, raw_away_prob
        draw = draw_weight

        total_goals_est = (home_attack + away_defense + away_attack + home_defense) / 2
        over = min(max(total_goals_est / 4.5, 0.1), 0.9)
        under = 1.0 - over

        return BaselineForecast(
            home_win=round(home_win, 4),
            draw=round(draw, 4),
            away_win=round(away_win, 4),
            over_2_5=round(over, 4),

```

```

        under_2_5=round(under, 4),
    )

    def _form_score(self, results: list[str]) -> float:
        """Normalise last-N results to a 0-1 form score."""
        if not results:
            return 0.5
        points = sum(self.RESULT_POINTS.get(r, 0) for r in results)
        return points / (len(results) * 3)

    def _renormalize_pair(self, first: float, second: float, total: float) -
> tuple[float, float]:
        """Scale a two-probability pair so it sums to the requested tota
l."""
        pair_total = first + second
        if pair_total == 0:
            return total / 2, total / 2
        scale = total / pair_total
        return first * scale, second * scale

```

```

In [5]: # -- Illustrative baseline run using the real MatchContext interface -----
from soccer_forecast_agent.models.match import (
    Match, MarketOdds, MatchContext,
)
from datetime import datetime, timedelta, timezone

strategy = SimpleBaselineStrategy(home_advantage_boost=0.05)

scenarios = [
    {
        "label": "Arsenal (strong) vs Brighton (weak)",
        "home": "Arsenal",
        "away": "Brighton",
        "home_results": ["W", "W", "W", "D", "W"],
        "away_results": ["L", "L", "D", "L", "L"],
        "home_goals_scored": 2.2,
        "home_goals_conceded": 0.8,
        "away_goals_scored": 0.9,
        "away_goals_conceded": 2.1,
    },
    {
        "label": "Everton (weak) vs Man City (strong)",
        "home": "Everton",
        "away": "Manchester City",
        "home_results": ["L", "L", "L", "D", "L"],
        "away_results": ["W", "W", "W", "W", "D"],
        "home_goals_scored": 0.8,
        "home_goals_conceded": 2.4,
        "away_goals_scored": 2.8,
        "away_goals_conceded": 0.6,
    },
    {
        "label": "Chelsea vs Tottenham (even)",
        "home": "Chelsea",
        "away": "Tottenham Hotspur",

```

```

        "home_results":      ["W", "D", "L", "W", "D"],
        "away_results":      ["D", "W", "L", "D", "W"],
        "home_goals_scored": 1.6,
        "home_goals_conceded": 1.4,
        "away_goals_scored": 1.5,
        "away_goals_conceded": 1.5,
    },
]

stub_odds = MarketOdds(
    odds_id="demo", match_id="demo",
    timestamp=datetime.now(timezone.utc),
    home_win=2.0, draw=3.4, away_win=3.8,
    over_2_5=1.9, under_2_5=1.9,
)

for s in scenarios:
    match = Match(
        match_id=s["home"] + "-vs-" + s["away"],
        competition="PL",
        home_team=s["home"],
        away_team=s["away"],
        kickoff_time=datetime.now(timezone.utc) + timedelta(days=1),
        status="upcoming",
    )
    context = MatchContext(
        match=match,
        odds=stub_odds,
        recent_home_results=s["home_results"],
        recent_away_results=s["away_results"],
        home_goals_scored_avg=s["home_goals_scored"],
        home_goals_conceded_avg=s["home_goals_conceded"],
        away_goals_scored_avg=s["away_goals_scored"],
        away_goals_conceded_avg=s["away_goals_conceded"],
    )
    result = strategy.compute(context)
    print()
    print(s["label"])
    print("  Home form :", s["home_results"])
    print("  Away form  :", s["away_results"])
    print("  Home win   :", f"{result.home_win:.1%}")
    print("  Draw       :", f"{result.draw:.1%}")
    print("  Away win   :", f"{result.away_win:.1%}")
    total = result.home_win + result.draw + result.away_win
    print("  Sum check  :", f"{total:.4f}")

```

Arsenal (strong) vs Brighton (weak)
Home form : ['W', 'W', 'W', 'D', 'W']
Away form : ['L', 'L', 'D', 'L', 'L']
Home win : 62.8%
Draw : 26.0%
Away win : 11.2%
Sum check : 1.0000

Everton (weak) vs Man City (strong)
Home form : ['L', 'L', 'L', 'D', 'L']
Away form : ['W', 'W', 'W', 'W', 'D']
Home win : 8.7%
Draw : 26.0%
Away win : 65.3%
Sum check : 1.0000

Chelsea vs Tottenham (even)
Home form : ['W', 'D', 'L', 'W', 'D']
Away form : ['D', 'W', 'L', 'D', 'W']
Home win : 39.6%
Draw : 26.0%
Away win : 34.4%
Sum check : 1.0000

```
In [6]: # — Live baseline run against real SQLite data —————
import sqlite3

from soccer_forecast_agent.config import Config
from soccer_forecast_agent.memory.sqlite_repository import SQLiteRepository
from soccer_forecast_agent.analytics.features import FeatureExtractor
from soccer_forecast_agent.agents.stats_market import StatsMarketAgent
from soccer_forecast_agent.domain.team_names import TeamNameNormalizer

config = Config.from_env()
conn = sqlite3.connect(config.db_path)
repo = SQLiteRepository(conn)
norm = TeamNameNormalizer()

all_teams = {
    row[0] for row in conn.execute("SELECT DISTINCT home_team FROM matches")
} | {
    row[0] for row in conn.execute("SELECT DISTINCT away_team FROM matches")
}
print(f"{len(all_teams)} teams found in SQLite:")
for t in sorted(all_teams):
    print(f" {t}")
```

23 teams found in SQLite:

AFC Bournemouth
Arsenal FC
Aston Villa FC
Brentford FC
Brighton & Hove Albion FC
Burnley FC
Chelsea FC
Crystal Palace FC
Everton FC
Fulham FC
Ipswich Town FC
Leeds United FC
Leicester City FC
Liverpool FC
Manchester City FC
Manchester United FC
Newcastle United FC
Nottingham Forest FC
Southampton FC
Sunderland AFC
Tottenham Hotspur FC
West Ham United FC
Wolverhampton Wanderers FC

```
In [7]: # — Run the baseline for two real fixtures from stored history —————

class _StubFetcher:
    def fetch_upcoming(self, *a, **kw): raise NotImplementedError
    def fetch_odds(self, *a, **kw): raise NotImplementedError

agent = StatsMarketAgent(
    fixture_fetcher=_StubFetcher(),
    odds_fetcher=_StubFetcher(),
    baseline_strategy=SimpleBaselineStrategy(home_advantage_boost=config.hon
    feature_extractor=FeatureExtractor(),
    match_repo=repo,
)

extractor = FeatureExtractor()

fixtures_to_inspect = [
    ("Arsenal", "Chelsea"),
    ("Manchester City", "Liverpool"),
]

placeholder_odds = MarketOdds(
    odds_id="demo", match_id="demo",
    timestamp=datetime.now(timezone.utc),
    home_win=2.1, draw=3.4, away_win=3.6,
    over_2_5=1.9, under_2_5=1.9,
)

for home_raw, away_raw in fixtures_to_inspect:
    home = norm.canonicalize(home_raw)
    away = norm.canonicalize(away_raw)
```

```

home_hist = repo.get_recent_finished(home, "PL", limit=5)
away_hist = repo.get_recent_finished(away, "PL", limit=5)

home_results = agent._results_for_team(home, home_hist)
away_results = agent._results_for_team(away, away_hist)
home_gs_list = agent._goals_scored_for_team(home, home_hist)
home_gc_list = agent._goals_conceded_for_team(home, home_hist)
away_gs_list = agent._goals_scored_for_team(away, away_hist)
away_gc_list = agent._goals_conceded_for_team(away, away_hist)

match = Match(
    match_id=f"{home}-vs-{away}",
    competition="PL",
    home_team=home,
    away_team=away,
    kickoff_time=datetime.now(timezone.utc) + timedelta(days=1),
    status="upcoming",
)
placeholder_odds.match_id = match.match_id

ctx = extractor.extract(
    match=match, odds=placeholder_odds,
    home_results=home_results, away_results=away_results,
    home_goals_scored=home_gs_list, home_goals_conceded=home_gc_list,
    away_goals_scored=away_gs_list, away_goals_conceded=away_gc_list,
)
bl = SimpleBaselineStrategy(home_advantage_boost=config.home_advantage_b

print(f"\n{'='*55}")
print(f" {home} vs {away}")
print(f"{'='*55}")
print(f" {home} recent : {home_results or ['no data']}")
print(f" {away} recent : {away_results or ['no data']}")
print(f" Home goals scored/conceded avg: {ctx.home_goals_scored_avg:.2f}")
print(f" Away goals scored/conceded avg: {ctx.away_goals_scored_avg:.2f}")
print()
print(f" Baseline forecast:")
print(f" Home win : {bl.home_win:.1%}")
print(f" Draw : {bl.draw:.1%}")
print(f" Away win : {bl.away_win:.1%}")
print(f" Over 2.5 : {bl.over_2_5:.1%}")
print(f" Under 2.5 : {bl.under_2_5:.1%}")
total = bl.home_win + bl.draw + bl.away_win
print(f" 1X2 sum : {total:.4f}")

```

```

=====
Arsenal vs Chelsea
=====
Arsenal recent : ['no data']
Chelsea recent : ['no data']
Home goals scored/conceded avg: 1.20 / 1.20
Away goals scored/conceded avg: 1.20 / 1.20

Baseline forecast:
Home win : 38.2%
Draw : 26.0%
Away win : 35.8%
Over 2.5 : 53.3%
Under 2.5 : 46.7%
1X2 sum : 1.0000

=====
Manchester City vs Liverpool
=====
Manchester City recent : ['no data']
Liverpool recent : ['no data']
Home goals scored/conceded avg: 1.20 / 1.20
Away goals scored/conceded avg: 1.20 / 1.20

Baseline forecast:
Home win : 38.2%
Draw : 26.0%
Away win : 35.8%
Over 2.5 : 53.3%
Under 2.5 : 46.7%
1X2 sum : 1.0000

```

Phase 3 — ReAct Research Agent

The `NewsContextAgent` runs a multi-step think-act-observe loop. It first seeds the loop with semantically relevant prior articles from ChromaDB, then uses live web search to gather breaking news, and finally uses the LLM to extract structured `EvidenceItem` objects from each observation.

```

In [8]: # — ReAct loop core logic (soccer_forecast_agent/agents/news_context.py) —
# Reproduced here for readability. Full implementation: soccer_forecast_agent

react_loop_pseudocode = """
def run(state):

    # 1. Seed with vector store context before loop starts
    seed_context = vector_repo.search(
        query=f"{home} vs {away} injuries suspensions form",
        teams=[home, away], top_k=5
    )
    evidence = extract_evidence(seed_context, source="vector_seed")

```

```

messages = build_prompt(match, baseline, seed_context, evidence)

# 2. ReAct loop
while not should_stop(evidence, step):
    step += 1

    # THINK: LLM reasons about what to search next
    thought, tool_name, tool_args = llm.chat_with_tools(messages, tools)

    if tool_name is None:          # LLM chose to stop
        break

    # ACT: call the tool
    observation = tool_dispatcher.dispatch(tool_name, tool_args)

    # OBSERVE: extract structured evidence from the result
    new_evidence = extract_evidence(observation, source=tool_name)
    evidence.extend(new_evidence)
    messages.append({"role": "tool", "content": observation})

return {**state, "evidence_items": evidence}

def should_stop(evidence, step):
    if step >= max_steps:          # budget exhausted
        return True
    if len(evidence) < min_count:  # not enough evidence yet
        return False
    avg_reliability = mean(e.reliability_score for e in evidence)
    return avg_reliability >= min_avg_reliability
"""

print(react_loop_pseudocode)

```



```

def run(state):

    # 1. Seed with vector store context before loop starts
    seed_context = vector_repo.search(
        query=f"{home} vs {away} injuries suspensions form",
        teams=[home, away], top_k=5
    )
    evidence = extract_evidence(seed_context, source="vector_seed")

    messages = build_prompt(match, baseline, seed_context, evidence)

    # 2. ReAct loop
    while not should_stop(evidence, step):
        step += 1

        # THINK: LLM reasons about what to search next
        thought, tool_name, tool_args = llm.chat_with_tools(messages, tools)

        if tool_name is None:          # LLM chose to stop
            break

        # ACT: call the tool
        observation = tool_dispatcher.dispatch(tool_name, tool_args)

        # OBSERVE: extract structured evidence from the result
        new_evidence = extract_evidence(observation, source=tool_name)
        evidence.extend(new_evidence)
        messages.append({"role": "tool", "content": observation})

    return {**state, "evidence_items": evidence}

def should_stop(evidence, step):
    if step >= max_steps:              # budget exhausted
        return True
    if len(evidence) < min_count:      # not enough evidence yet
        return False
    avg_reliability = mean(e.reliability_score for e in evidence)
    return avg_reliability >= min_avg_reliability

```

```

In [9]: # — Structured evidence output – real source —————
print("=== EvidenceItem model ===\n")
print(inspect.getsource(EvidenceItem))
print()

# Illustrative example of what the agent produces
from datetime import datetime, timezone
from uuid import uuid4

example_evidence = EvidenceItem(
    evidence_id=str(uuid4()),
    forecast_id="demo-forecast-001",
    source="bbc.co.uk",
    url="https://www.bbc.co.uk/sport/football/example",
    timestamp=datetime.now(timezone.utc),

```

```

        summary="Arsenal's starting striker is ruled out with a hamstring injury
        direction="away_positive",
        reliability_score=0.85,
        applies_to_market="winner",
    )

    print("Example EvidenceItem:")
    print(f" Source      : {example_evidence.source}")
    print(f" Summary      : {example_evidence.summary}")
    print(f" Direction      : {example_evidence.direction}")
    print(f" Reliability    : {example_evidence.reliability_score}")
    print(f" Market        : {example_evidence.applies_to_market}")

```

=== EvidenceItem model ===

```

@dataclass
class EvidenceItem:
    """A single piece of qualitative evidence gathered by the ReAct research
    agent."""

    evidence_id: str
    forecast_id: str
    source: str
    url: str
    timestamp: datetime
    summary: str
    direction: str          # "home_positive" | "away_positive" | "neutral" |
    "uncertainty"
    reliability_score: float # 0.0 – 1.0
    applies_to_market: str   # "winner" | "goals" | "both"

```

Example EvidenceItem:

```

Source      : bbc.co.uk
Summary      : Arsenal's starting striker is ruled out with a hamstring inj
ury ahead of the weekend fixture.
Direction     : away_positive
Reliability   : 0.85
Market        : winner

```

```

In [10]: # — Log-odds adjustment mechanism (Phase 4 – SynthesisAlertAgent) —
# Implemented as helpers; full run() wiring is Phase 4.
import math

def sigmoid(x: float) -> float:
    return 1.0 / (1.0 + math.exp(-x))

def adjust_probability(
    baseline_prob: float,
    evidence_items: list,
    base_sensitivity: float = 0.15,
) -> float:
    """Shift a probability in log-odds space using evidence deltas."""
    DIRECTION_SIGN = {"home_positive": 1, "away_positive": -1, "neutral": 0,
    MARKET_WEIGHT = {"both": 1.0, "winner": 0.7, "goals": 0.7}

```

```

log_odds = math.log(baseline_prob / (1 - baseline_prob))

total_delta = sum(
    DIRECTION_SIGN.get(e.direction, 0)
    * e.reliability_score
    * MARKET_WEIGHT.get(e.applies_to_market, 0.7)
    * base_sensitivity
    for e in evidence_items
)

return sigmoid(log_odds + total_delta)

# Demo: apply the example evidence item to a baseline probability
baseline_home_win = 0.52 # 52% home win from statistical baseline

adjusted = adjust_probability(
    baseline_prob=baseline_home_win,
    evidence_items=[example_evidence],
    base_sensitivity=0.15,
)

print(f"Baseline home-win probability : {baseline_home_win:.1%}")
print(f"Evidence                     : {example_evidence.direction} (key st
print(f"Adjusted home-win probability : {adjusted:.1%}")
print(f"Shift                        : {(adjusted - baseline_home_win):+.1%

```

```

Baseline home-win probability : 52.0%
Evidence                     : away_positive (key striker injured)
Adjusted home-win probability : 49.8%
Shift                        : -2.2%

```

Test Suite

All implemented components have unit tests. Tests use stub/fake implementations rather than mocks so behavior — not implementation detail — is validated.

```

In [11]: import subprocess, sys, os

result = subprocess.run(
    [sys.executable, "-m", "pytest", "tests/",
     "-v", "--tb=short",
     "--ignore=tests/test_live_llm_provider.py",
     "--ignore=tests/test_live_news_context.py"],
    capture_output=True, text=True,
    cwd=os.path.abspath("."))
print(result.stdout)
if result.returncode != 0:
    print(result.stderr)

```

```
===== test session starts =====
=====
platform darwin -- Python 3.11.15, pytest-9.0.3, pluggy-1.6.0 -- /Users/lize
thbuendia/Desktop/SoccerForecastAgent/.venv/bin/python
cachedir: .pytest_cache
rootdir: /Users/lizethbuendia/Desktop/SoccerForecastAgent
configfile: pyproject.toml
plugins: langsmith-0.7.36, anyio-4.13.0
collecting ... collected 35 items

tests/test_alert_guard.py::test_alert_guard_passes_with_recent_diverse_relia
ble_evidence PASSED [ 2%]
tests/test_alert_guard.py::test_alert_guard_collects_multiple_failure_reason
s PASSED [ 5%]
tests/test_alert_guard.py::test_alert_guard_handles_naive_datetimes_without_
crashing PASSED [ 8%]
tests/test_alert_guard.py::test_alert_guard_uses_default_config_values PASSE
D [ 11%]
tests/test_baseline.py::test_simple_baseline_returns_normalized_probabilitie
s PASSED [ 14%]
tests/test_baseline.py::test_simple_baseline_favors_stronger_home_side PASSE
D [ 17%]
tests/test_baseline.py::test_simple_baseline_keeps_nonzero_away_win_probabil
ity_for_weak_side PASSED [ 20%]
tests/test_chroma_repository.py::test_chroma_repository_uses_embedding_provi
der_for_upsert_and_query PASSED [ 22%]
tests/test_config.py::test_config_defaults_embedding_provider_and_model PASS
ED [ 25%]
tests/test_config.py::test_config_reads_llm_model_override PASSED [ 28%]
tests/test_embedding_retrieval_fixture.py::test_sample_fixture_contains_six_
articles PASSED [ 31%]
tests/test_embedding_retrieval_fixture.py::test_retrieval_fixture_surfaces_e
xpected_team_and_topic_matches PASSED [ 34%]
tests/test_features.py::test_feature_extractor_builds_context_with_last_five
_results PASSED [ 37%]
tests/test_features.py::test_feature_extractor_uses_default_average_for_empt
y_series PASSED [ 40%]
tests/test_ingester.py::test_article_ingester_skips_blank_and_duplicate_urls
PASSED [ 42%]
tests/test_ingester.py::test_article_ingester_chunking_uses_overlap PASSED [
45%]
tests/test_main.py::test_local_mcp_tool_client_dispatches_search_news PASSED
[ 48%]
tests/test_main.py::test_local_mcp_tool_client_dispatches_odds_and_fixtures
PASSED [ 51%]
tests/test_news_context.py::test_tool_dispatcher_routes_calls_and_serialises
_results PASSED [ 54%]
tests/test_news_context.py::test_seed_context_queries_vector_repo_for_both_t
eams PASSED [ 57%]
tests/test_news_context.py::test_should_stop_requires_enough_high_reliabilit
y_evidence PASSED [ 60%]
tests/test_news_context.py::test_news_context_agent_run_collects_and_saves_e
vidence PASSED [ 62%]
tests/test_news_context.py::test_news_context_agent_ignores_malformed_eviden
ce_json PASSED [ 65%]
```

```

tests/test_news_context.py::test_news_context_agent_records_unknown_tool_err
or PASSED [ 68%]
tests/test_news_context.py::test_news_context_agent_records_evidence_save_fa
ilures PASSED [ 71%]
tests/test_prompt_loader.py::test_render_news_context_research_messages_incl
udes_dynamic_fields PASSED [ 74%]
tests/test_prompt_loader.py::test_render_news_context_evidence_messages_incl
udes_constraints PASSED [ 77%]
tests/test_sqlite_repository.py::test_sqlite_repository_round_trips_match_fo
recast_and_evidence PASSED [ 80%]
tests/test_sqlite_repository.py::test_sqlite_repository_updates_match_status
_and_uses_seeded_source_scores PASSED [ 82%]
tests/test_sqlite_repository.py::test_sqlite_repository_returns_recent_finis
hed_matches_for_team PASSED [ 85%]
tests/test_stats_market_agent.py::test_stats_market_agent_populates_matches_
odds_and_baselines PASSED [ 88%]
tests/test_stats_market_agent.py::test_stats_market_agent_records_error_when
_odds_missing PASSED [ 91%]
tests/test_stats_market_agent.py::test_stats_market_agent_uses_recent_finish
ed_history_for_baseline PASSED [ 94%]
tests/test_team_name_normalizer.py::test_team_name_normalizer_maps_known_ali
ases_to_canonical_names PASSED [ 97%]
tests/test_team_name_normalizer.py::test_team_name_normalizer_returns_trimme
d_input_for_unknown_name PASSED [100%]

===== 35 passed in 6.03s =====
=====

```

Roadmap

Phase	Description	Status
0	Final design, schema, guardrail thresholds	✓ Complete
1	Core infrastructure — models, protocols, tools, providers	✓ Complete
2	Baseline analytics — form/goals strategy, StatsMarketAgent	✓ Complete
3	ReAct research agent — NewsContextAgent, dual retrieval, evidence extraction	✓ Complete
4	Synthesis and alerts — log-odds adjustment, SynthesisAlertAgent.run()	🔄 In progress
5	Orchestration — LangGraph graph, SupervisorAgent, evaluation pipeline	■ Planned
6	Final submission packaging — progress notebook cleanup, export, and demo presentation	■ Planned

Phase	Description	Status
7	Advanced analytics — Dixon-Coles model (stretch goal)	Stretch

Progress report — core infrastructure, baseline analytics, and news-context research components are implemented. Source code is included in `soccer_forecast_agent/`, with full end-to-end orchestration still in progress.